Testing the Lake Temperature Model

Purpose

- Find and identify patterns in how changing the constants of elmtest.exe, an environmental model, affects its outputs.

What is elmtest.exe?

- Fortran binary
- Compiled in a docker image from Docker4SPEL
- Environmental model

elmtest.exe interface

- Inputs: lakeparams.txt
- Outputs: cpu_LakeTemperatureTest.txt

lakeparams.txt

CONTAINER:/app/elmtest/lakeparams.txt

```
betavis
1.0
za_lake
2.0
n2min
7.5e-05
tdmax
275.0
pudz
11.0
mixfact
0.5
depthcrit
25.0
```

cpu_LakeTemperatureTestDocker.txt

CONTAINER:/app/elmtest/cpu_LakeTemperatureTestDocker.txt (Default Parameters)

```
col_ws%h2osno
   0.000000000000000        0.000000000000000        0.000000000000000
   0.000000000000000        0.000000000000000        0.000000000000000
   0.000000000000000        0.000000000000000        0.000000000000000
   0.000000000000000        0.000000000000000        0.000000000000000
   0.000000000000000        0.000000000000000        0.000000000000000
   0.000000000000000        0.000000000000000        0.000000000000000
                                   - - - … - - -

col_es%hc_soi
  1.0000000000000000E+036   1.0000000000000000E+036   1.0000000000000000E+036
  1.0000000000000000E+036   1.0000000000000000E+036   1.0000000000000000E+036
  1.0000000000000000E+036   1.0000000000000000E+036   1.0000000000000000E+036
  1.0000000000000000E+036   1.0000000000000000E+036   1.0000000000000000E+036
  1.0000000000000000E+036   1.0000000000000000E+036   1.0000000000000000E+036
  1.0000000000000000E+036    15270.67984628235        1.0000000000000000E+036
                                   - - - … - - -
```

Development using fake_binary.exe

- The Docker container is stateless
- elmtest.exe depends on the environment inside the docker image
- fake_binary.exe was developed to use the same interface as elmtest.exe but without depending on the docker image
  - Uses known functions to compute output, making testing easier

fake_LakeTemperatureTest.txt

lake_temp_work/lake_temperature_mapper/fake_binary/fake_LakeTemperatureTest.txt
(Default Parameters)

```
fakevar%sum
  314.500075 629.000150
  31.450007 157.250037
fakevar%product
  31.450007 157.250037
  5.671875 32.170166
```

Original Testing Approach

- Generate input values using a json file with instructions for linear interpolation
  - Became the Order Sampling plugin
- Write change to lakeparams.txt
- Run elmtest.exe
- Compare cpu_LakeTemperatureTest.txt to cpu_LakeTemperatureRef.txt
  - The difference analysis performed here became the Differences analysis plugin
- Repeat

What prompted a new approach?

- Alternate sampling methods
  - CSV of indices from NIST’s ACTS
- Alternate analysis
  - Linear Regression
  - Overall variance
  - Find physically impossible outputs

New Approach

- Split testing program into three steps
  - Sample Generation
    - Generate input values to give to the model
  - Testing
    - Run the model
  - Analysis
    - Analyze the resulting data

Sample Generation

- What is a sample?
  - A dictionary mapping input parameter names to their values
  - Example:

```
{ betavis: 1.0, za_lake: 2.0, n2min: 7.5e-05, tdmax: 275.0, pudz: 11.0, … }
```

Sample Generation

- What is a sample group?
  - A named list of samples
  - Intended to group and order similar samples
  - As an example, 11 samples interpolating betavis through the range [0.0, 1.0] could be a sample group

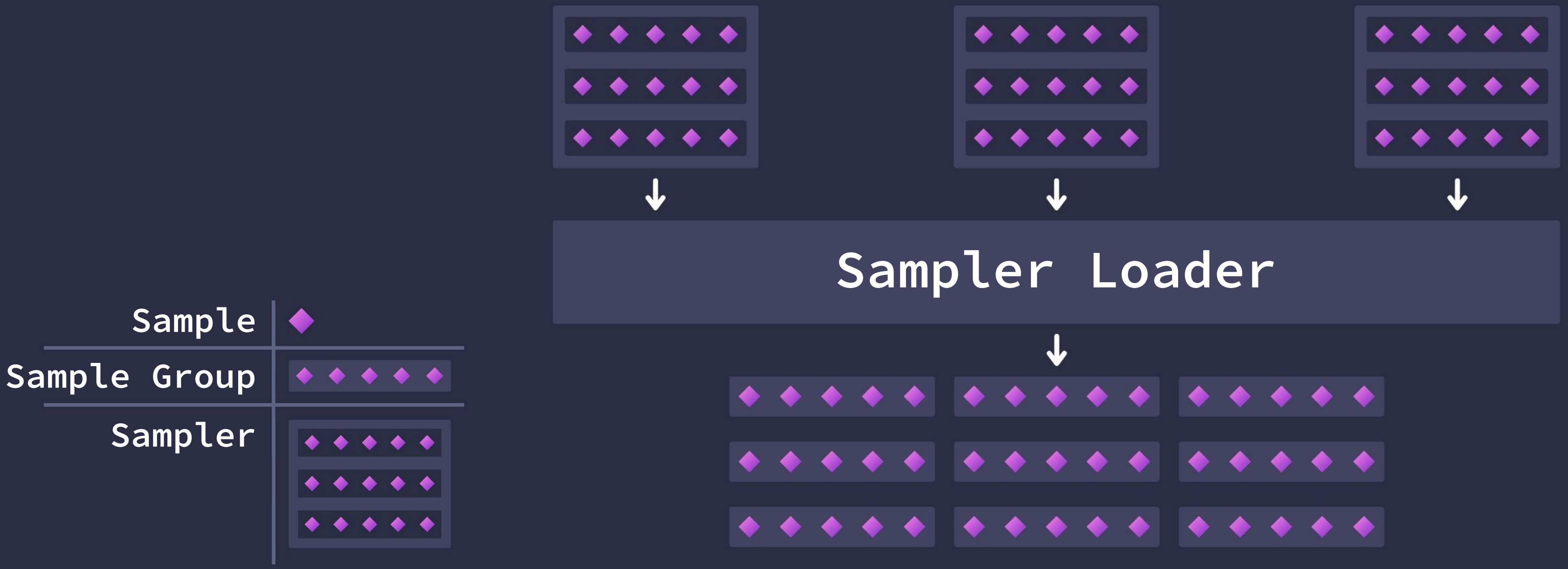
Sample Generation

- What is a sampler?
  - A class that generates any number of sample groups
  - Acts as the interface between a sampling plugin and the testing program
    - A sampling plugin must have one class extending Sampler

Sample Generation

- What is a sampler loader?
  - The class that discovers and loads sampling plugins.
  - It then collects a list of samplers from each plugin, and instantiates them. (Loading)
  - Then it requests all of the sample groups from each sampler. (Sampling)

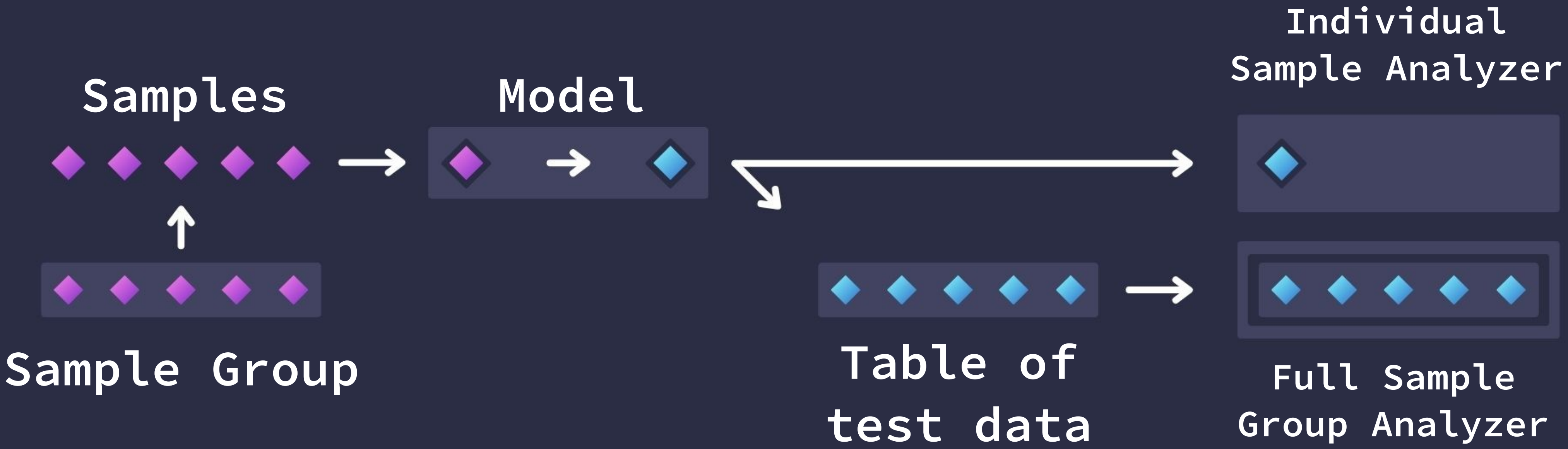
Sample Generation



Testing

- Write a sample to the model’s parameters file
- Run the model
- Read the model’s output file
- Send that test data to individual sample analyzers
- When a full sample group has been tested, send a table containing each sample’s test data to full sample group analyzers

Testing


Individual
Sample Analyzer
Samples
Model
Sample Group
Table of
test data
Full Sample
Group Analyzer

Analysis

- All plugins discovered and loaded by Analyzer Loader
- Plugins can either analyze samples individually or as a group.
  - Contains a PerSampleAnalyzer subclass if opting to analyze samples individually
  - Contains a SampleGroupAnalyzer subclass if opting to analyze samples as a group
- Each plugin is given it’s test data, the reference data, and the samples used to generate that data

Extras

Order Sampler (Sampling Plugin)

- Generates samples using “order files”
  - JSON files with instructions for linearly interpolating through ranges for each input parameter
- Each order file is treated as its own sample group

Example Order: betavis_min_to_max.json

lake_temp_work/lake_temperature_mapper/mapping_orders/betavis_min_to_max.json

```
{
  "samples":11,
  "ranges":{
    "betavis": ["r(0)", "r(1)"]
  }
}
```

Results in: [0.0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0]

What does r(t) mean?
Represents a value returned from performing an unbounded linear interpolation between the minimum and maximum size specified in the range file (FUT_lake_range.txt for Lake Temperature), with t as the time value.

Example Line Order: all_min_to_max.json

lake_temp_work/lake_temperature_mapper/mapping_orders/all_min_to_max.json

```
{
  "samples":11,
  "ranges": {
    "betavis":["r(0)", "r(1)"],
    "za_lake":["r(0)", "r(1)"],
    "n2min":["r(0)", "r(1)"],
    "tdmax":["r(0)", "r(1)"],
    "pudz":["r(0)", "r(1)"],
    "mixfact":["r(0)", "r(1)"],
    "depthcrit":["r(0)", "r(1)"]
  }
}
```

Interpolates between each range *simultaneously,* forming a line.

Example Box Order: betavis_za_lake_box.json

lake_temp_work/lake_temperature_mapper/mapping_orders/betavis_za_lake_box.json

```
{
  "ranges": {
    "betavis": ["r(0)", "r(1)", 5],
    "za_lake": ["r(0)", "r(1)", 5]
  }
}
```

Interpolates between each axis, treating each line as a nested for loop.
Samples will form a multi-dimensional box.